Министерство образования Республики Беларусь
Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Конструирование программного обеспечения»

Тема: «Настройка взаимодействия с файлами»

Выполнил:

Студент 2 курса

Группы ПО-17

Ковалев А.И.

Проверил:

Кулик А.Д.

Брест 2026

Цель работы

Настроить взаимодействие проекта из первой лабораторной работы с файлами: сделать так, чтобы настройки игры записывались в файл и загружались из него при запуске программы.

Условия выполнения

Код программы писал в онлайн-компиляторе, локально среду разработки не ставил. Итоговый код лежит в файле src/main.cpp, тесты в файле tests/test_main.cpp, а скрипт tests/run_tests.sh собирает тесты через g++ и запускает их.

Задание

Нужно добавить в программу сохранение и загрузку настроек из файла. При первом запуске должен создаваться файл settings с настройками по умолчанию, а запись в него должна происходить при изменении любой настройки. По варианту 3 уровень сложности хранится в переменной типа string, а информация о животных в контейнере Map. Кроме того, в настройках должна появиться возможность добавить новый вид животного, который сразу записывается в файл вместе с количеством.

Описание программы

За основу взята программа из первой лабораторной работы, меню и пункты остались прежними, версия в пункте «О программе» теперь 2.0. Главное изменение в том, как хранятся настройки:

```
string settingsFile = "settings.txt";
string difficulty = DEFAULT_DIFFICULTY;
map<string, int> animals;
```

Сложность теперь строка (Легко, Нормально или Сложно), а животные лежат в контейнере map, где ключ это название вида, а значение количество. Map удобен тем, что в нём нет повторяющихся названий и можно быстро найти животное по имени. Имя файла вынесено в переменную settingsFile, чтобы тесты могли подставить другой файл и не портить настоящие настройки.

Запись настроек выполняет функция saveSettings():

```
ofstream out(settingsFile.c_str());
out << "difficulty=" << difficulty << "\n";
for (const auto& animal : animals)
    out << ANIMAL_PREFIX << animal.first
        << "=" << animal.second << "\n";
```

Объект ofstream открывает файл для записи, а дальше в него пишут так же, как в cout. Цикл проходит по всем животным из map и записывает каждое отдельной строкой. Файл при этом получается таким:

```
difficulty=Нормально
animal.Корова=3
animal.Курица=5
```

Загрузка выполняется функцией loadSettings(), которая вызывается в начале main(). Сначала она ставит настройки по умолчанию, потом пробует открыть файл:

```
ifstream in(settingsFile.c_str());
if (!in.is_open())
{
    saveSettings();
    return;
}
```

Если файл не открылся, значит это первый запуск. Тогда сразу вызывается saveSettings(), и файл создаётся с настройками по умолчанию (3 коровы и 5 кур, сложность «Нормально»). Если файл есть, он читается построчно через getline. Каждая строка делится по знаку равно на ключ и значение. Если ключ начинается с animal., то это животное, и его количество превращается в число:

```
try
{
    size_t used = 0;
    int count = stoi(value, &used);
    if (used != value.size() || count < MIN_COUNT
        || count > MAX_COUNT)
        throw out_of_range("count");
    loadedAnimals[key.substr(ANIMAL_PREFIX.size())] = count;
}
catch (const exception&)
{
    cout << "Предупреждение: пропущена неверная строка...";
}
```

Функция stoi выбрасывает исключение, если в строке не число. Исключение выбрасывается и вручную, если после числа остались лишние символы или число вне диапазона от 1 до 100. Блок catch перехватывает любую такую ошибку, программа выводит предупреждение и просто пропускает испорченную строку. Если в файле были ошибки, он после загрузки перезаписывается уже исправленным.

Запись в файл происходит при любом изменении настроек. Функция saveSettings() вызывается в конце changeDifficulty(), changeAnimalCount() и addAnimal(). Новый вид животного добавляется так:

```
if (animals.count(name) > 0)
{
    cout << "Ошибка: такой вид уже есть.\n";
    return;
}
...
animals[name] = count;
saveSettings();
```

Сначала проверяется, что такого названия ещё нет (count возвращает 1, если ключ уже есть в map). Потом количество записывается в map, и сразу после этого весь файл перезаписывается, поэтому новый вид оказывается в файле в тот же момент.

Меню настроек теперь содержит три пункта: уровень сложности, количество животных и добавление нового вида. В пункте «Количество животных» выводится список всех видов из map, пользователь выбирает номер и вводит новое количество от 1 до 100. При старте игры показывается сложность и все животные из контейнера. Остальные части (readInt, waitForInput, главное меню и main) остались такими же, как в первой лабораторной.

Тестирование

Тесты, как и раньше, написаны на C++ в файле tests/test_main.cpp. Они подключают код программы, подменяют ввод и вывод на строки и проверяют вывод и значения переменных. Новое в этой работе то, что тесты работают с настоящим файлом: перед каждой проверкой тест сам создаёт, портит или удаляет тестовый файл build/test_settings.txt, поэтому настоящие настройки не затрагиваются.

Всего написано 45 проверок. Они покрывают пункты главного меню, первый запуск (файл создаётся и в нём лежат значения по умолчанию), запись в файл после смены сложности и количества животных, добавление нового вида и его запись в файл, а также отказ при повторном названии, пустом названии и неверном количестве. Отдельно проверено, что настройки не сбрасываются после перезапуска: тест меняет настройки, запускает программу второй раз и убеждается, что значения загрузились из файла. Проверены и неправильные файлы: с мусорными строками, пустой и с переводами строк Windows. Все тесты прошли успешно. Чтобы убедиться, что тесты что-то проверяют, в код специально вносились ошибки (убиралась запись в файл, не создавался файл при первом запуске), и тесты в этих случаях выдавали FAIL.

Контрольные вопросы

Алгоритм работы написанной программы

При запуске вызывается loadSettings(): если файла настроек нет, он создаётся с настройками по умолчанию, а если есть, настройки читаются из него. Дальше работает цикл главного меню, как в первой лабораторной. Пункты «Начать игру», «Загрузить игру» и «О программе» выводят сообщение и ждут Enter. В «Настройках» можно поменять сложность, количество животных или добавить новый вид, и после каждого изменения файл перезаписывается. Программа заканчивается, когда выбран пункт «Выход».

Библиотеки файлового ввода и вывода

В C++ для работы с файлами используется библиотека fstream. В ней есть ifstream для чтения, ofstream для записи и fstream для обоих действий. С файловыми потоками работают так же, как с cin и cout: операторами << и >> и функцией getline. Файл можно открывать в разных режимах (чтение, запись, дозапись в конец, двоичный режим), а то, что он открылся, проверяют методом is_open(). Закрывается файл автоматически, когда объект потока уничтожается. В языке Си для этого есть библиотека cstdio с функциями fopen, fprintf и

fclose, а в C++17 появилась библиотека filesystem для работы с путями и проверки существования файлов. В моей работе использованы ifstream и ofstream.

Контейнеры

Контейнеры это объекты, которые хранят набор элементов. Последовательные контейнеры хранят элементы по порядку: vector, list, deque. Ассоциативные контейнеры: set хранит уникальные значения, а map хранит пары «ключ и значение», причём элементы упорядочены по ключу. Есть и варианты unordered_set и unordered_map на основе хеш-таблицы, они обычно быстрее. Контейнеры-адаптеры stack (последним пришёл, первым ушёл), queue (первым пришёл, первым ушёл) и priority_queue построены поверх других контейнеров. В работе использован map<string, int> для животных и vector для списка названий в меню.

Логирование

Логирование это запись сообщений о работе программы в файл или на экран: что запустилось, какие действия выполнялись, где возникли предупреждения и ошибки. Обычно к сообщению добавляют время и уровень важности (INFO, WARNING, ERROR, DEBUG). Лог помогает найти причину сбоя уже после того, как он случился, особенно если программа работает у пользователя. В моей программе отдельного файла журнала нет, но предупреждения о неверных строках в файле настроек выводятся на экран.

Unit-тестирование

Unit-тестирование это проверка отдельных небольших частей программы (функций, модулей) автоматическими тестами. Каждый тест подаёт на вход известные данные и сравнивает результат с ожидаемым. Такие тесты позволяют быстро заметить, что после изменения кода что-то сломалось. Для этого есть готовые фреймворки, например Google Test и Catch2, но можно обойтись и без них. В работе тесты написаны без фреймворка с помощью собственной функции check.

Обработка исключений

Исключения это механизм обработки ошибок, которые возникают во время работы программы. Код, который может вызвать ошибку, помещают в блок try, ошибку создают оператором throw, а блок catch её перехватывает, и программа не завершается аварийно. Стандартные исключения, например invalid_argument и out_of_range, наследуются от класса exception. В работе исключения используются при чтении файла настроек: stoi выбрасывает исключение, если в строке не число, а catch пропускает такую строку с предупреждением.

Вывод

В ходе работы программа из первой лабораторной была доработана: настройки теперь хранятся в файле settings, создаются при первом запуске, записываются при каждом

изменении и загружаются при старте, поэтому после закрытия программы больше не сбрасываются. Я использовал файловые потоки ifstream и ofstream, контейнер map, обработку исключений для защиты от испорченного файла и добавил возможность создавать новые виды животных. Для проверки написаны 45 автоматических тестов, все они проходят.